ADVANCED WORKSHOP

From agent user to agent orchestrator

- GitHub Copilot / custom agents / skills / safe parallel work
- 3 hours: 60 minutes together + 120 minutes building
- Official guidance verified 7 November 2026 (US Eastern)

START HERE

What you will be able to do

- Choose instructions, a custom agent, or a skill for the right job
- Delegate a bounded task with an evidence-based result
- Coordinate one repository without overlapping writers
- Review and integrate changes instead of trusting completion messages

RUN OF SHOW

A realistic three-hour agenda

- 00:00-00:15 Concepts + app and VS Code tour
- 00:15-00:35 Agent / skill creation + delegation demos
- 00:35-01:00 Safe orchestration + review demo
- 01:00-03:00 Labs: 30 + 35 + 40 + 15 minutes

MENTAL MODEL

Five words that prevent confusion

- Model: generates and reasons about possible next actions
- Harness: manages context, tool calls, approvals and execution
- Custom agent: reusable role + instructions + selected tools
- Subagent: delegated work in a separate context
- Skill: an on-demand procedure with supporting resources

Sources: S5, S8, S9

IMPORTANT DISTINCTION

Separate context is not separate files

- Two chats can point at the same checkout
- Subagents do not automatically receive independent worktrees
- A worktree separates working files, not machine permissions
- An approval mode is not a filesystem lock

Sources: S4, S7, S15

FRESHNESS

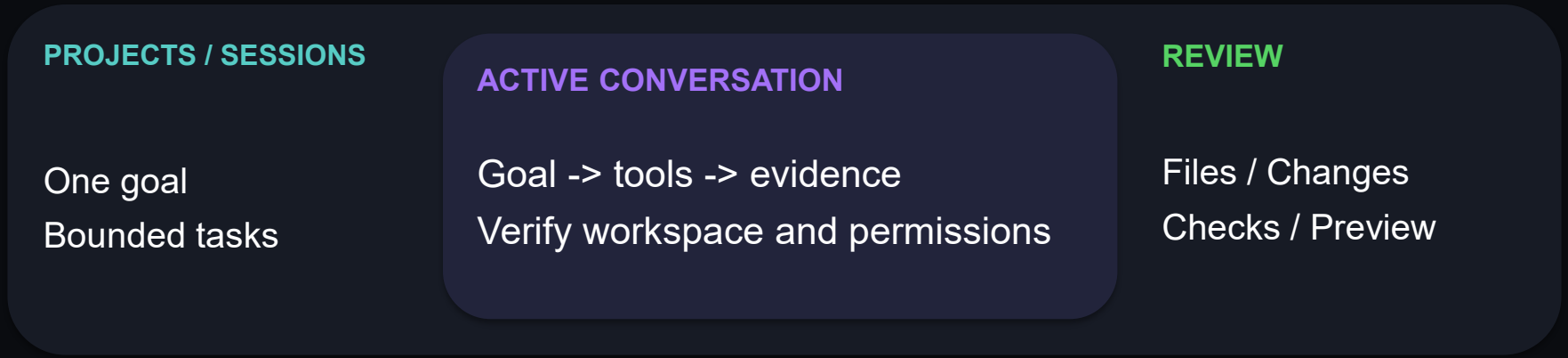
Teach the documented product, not old defaults

- Current documentation checked: 7 November 2026
- 28 October release: Copilot harness default for new VS Code sessions
- App Projects, marketplace and connectors: preview-dependent
- Core labs avoid preview-only functionality

Sources: S1, S2, S13, S14

GITHUB COPILOT APP

A desktop home for agent-driven work



Original orientation schematic; use the live product for exact controls.

Choose New worktree or Current checkout explicitly / Review changes, GitHub issues, PRs and checks in one place

Sources: S1, S2, S3

APP DEMONSTRATION

Direct one small task, then inspect it

- Select the starter repository and verify workspace
- Use Plan: outcome + constraints + acceptance checks
- Approve a bounded change with appropriate permissions
- Inspect the complete diff; rerun checks before integration

Sources: S3, S12

VS CODE AGENTS WINDOW

An agent-first window beside your editor

SESSIONS / CUSTOMIZATIONS

One goal

Bounded tasks

ACTIVE CONVERSATION

Goal -> tools -> evidence

Verify workspace and permissions

REVIEW

Files / Changes

Checks / Preview

Original orientation schematic; use the live product for exact controls.

Or use the command line: code --agents / Confirm the active pane before commands or feedback

Sources: S4, S13

COMPATIBILITY

Select the harness before copying YAML

- Copilot: native task delegation; skills field supported
- Local: agent/runSubagent; agents allowlist + handoffs
- Local does not accept the Copilot skills field
- target selects file interpretation, not the running harness

Sources: S5, S6, S7

Write a task contract, not a wish

- Goal and concrete acceptance examples
- Relevant files, constraints and non-goals
- Allowed actions and explicit write ownership
- Tests + evidence + a stopping condition

```
GOAL: implement one named behavior
CONTEXT: contract + relevant files + examples
WRITE SET: module A + tests A only
CHECKS: exact commands and expected results
RETURN: evidence, risks, blocked decisions
STOP: no unrelated improvements
```

Sources: S7, S12

Put shared rules where everyone can find them

- .github/copilot-instructions.md: repository-wide guidance
- .github/instructions/*.instructions.md: scoped guidance
- AGENTS.md: support and activation vary by experience
- Pass important rules explicitly to delegated workers

Sources: S9, S12, S14

CUSTOM AGENT

A reviewer that can read, not write

- A focused role beats a grandiose persona
- Use only tools the role needs
- Define useful findings and honest uncertainty

```
---
name: Workshop Reviewer
description: Review behavior and missing tests
tools: ['read', 'search']
---
Read the assigned source and tests.
Do not edit or run commands.
Return file, issue, impact and suggested test.
State what you could not verify.
```

Sources: S5, S6

A reusable recipe with a trigger

- Folder: .github/skills/test-changes/
- Required: SKILL.md name + description
- Name matches folder; describe WHAT and WHEN
- Resources and scripts stay with the skill

```
---
name: test-changes
description: Validate code changes with the
  repository tests; use before a handoff.
---
1. Read the repository test commands.
2. Run checks for the assigned change.
3. Report passed / failed / not run.
Use [report format](./references/report.md).
```

Sources: S8

Discovery is not execution evidence

- Metadata first; full procedure when relevant
- References and scripts loaded only as needed
- Test positive, negative and ambiguous requests
- No automatic grants of credentials or tools

Sources: S8, S12

Compose them without mixing harnesses

- Copilot agent: skills: ['test-changes']
- Local agent: ask it to load the relevant skill
- Both: narrow tools and explicit success criteria
- Verify the enabled skill inventory in a fresh session

```
# Copilot harness example
---
name: Bounded Implementer
description: Implement one assigned change
target: github-copilot
tools: ['read', 'search', 'edit', 'execute']
skills: ['test-changes']
---
Stay in the assigned write set.
Return validation evidence and risks.
```

Sources: S5, S8

DEMO 1

Build and verify a role and a recipe

- Create the reviewer; confirm tool restrictions
- Create a skill with a real report reference
- Use an explicit invocation, then a matching plain-language request
- Show one skipped check reported honestly

Sources: S6, S8

DELEGATION

Use a subagent for a bounded result

- Good: inspect a module and return patterns + unknowns
- Good: implement one independent owned task
- Poor: split a tiny edit or delegate an undefined goal
- Cost grows with calls, context and coordination

Sources: S7, S9, S10

DELEGATED BRIEF

Give the worker enough context to succeed

- Exact goal and relevant contract version
- Allowed files and prohibited shared files
- Acceptance examples and check commands
- Return findings, evidence, unknowns and stop

```
Use a read-only researcher subagent.
Goal: locate current triage behavior.
Context: fictional issues; no dependencies.
Read: contract, rules module and tests.
Do not edit or execute commands.
Return paths, I/O shape, existing patterns,
unknowns and what you did not verify.
Stop after this report.
```

Sources: S7, S9

Named workers, one implementation writer

- Local-only agents list limits delegation
- Researcher and reviewer remain read-only
- Coordinator makes the implementation change
- Re-invoke review with updated context when needed

```
---
name: Local Coordinator
description: Research, implement and review
target: vscode
tools: ['agent', 'read', 'search', 'edit',
'execute']
agents: ['Researcher', 'Workshop Reviewer']
---
Delegate read-only research; implement once.
Validate, then delegate read-only review.
```

Sources: S5, S7

Same workflow, different runtime contract

- Use native delegation through the agent alias
- No Local agents allowlist or handoffs fields
- Pass skill, constraints and expected result explicitly
- /fleet is optional, not automatic file isolation

```
---
name: Copilot Coordinator
description: Coordinate bounded triage work
target: github-copilot
tools: ['agent', 'read', 'search', 'edit',
'execute']
skills: ['test-changes']
---
Pass self-contained briefs to specialists.
Keep one writer per shared file.
Review and validate before integrating.
```

Sources: S5, S9, S10

Switching roles is not parallel delegation

- Handoff: Local UI transition to the next agent
- Subagent: main agent delegates and keeps ownership
- Separate session: independent tracked work and workspace choice
- Fallback: manually paste a concise reviewed handoff

```
# Local-only optional frontmatter
handoffs:
  - label: Implement reviewed plan
    agent: agent
    prompt: Implement the reviewed plan.
    send: false
```

Sources: S5, S6, S7

DEMO 2

Research -> implement -> prove -> review

- Research the issue input and rule boundary
- ISS-101 -> score 100 / P1; closed issues -> 0 / P3
- Nearest reachable P1 boundary: 75 / P2 vs 85 / P1
- Review source evidence, then fix or report findings

Sources: S7, S12

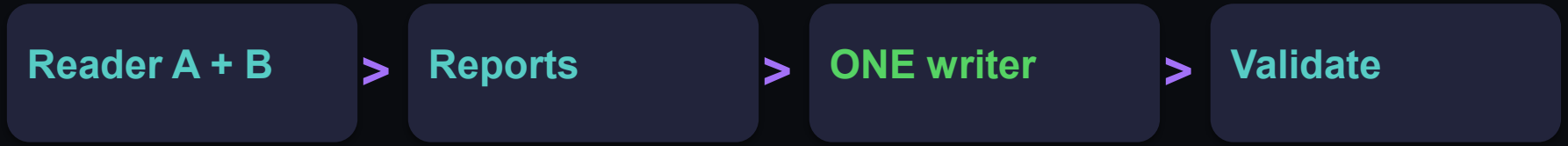
Hot files and shared resources

- Shared contracts, package manifests and lockfiles
- Server wiring, generated outputs and fixtures
- Git index, branch switching and concurrent commits
- Ports, databases, credentials and external side effects

Sources: S4, S15

SAFE DEFAULT

Parallel readers, one integration writer



- Workers return findings or proposed patches
- One integrator applies edits and owns shared files
- Combine results only after checking their current base

Sources: S7, S10

FILE OWNERSHIP

Freeze the contract, then split the work

WORKER A	Owned module A + tests A
WORKER B	Owned module B + tests B
INTEGRATOR	Contract / UI wiring / fixtures / Git

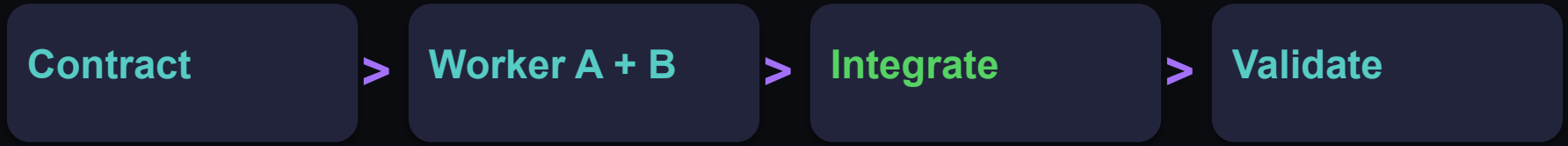
Freeze the contract. Stop before touching another owner's file.

Ownership is advisory; validate the actual changed paths.

Sources: S4, S10

DEPENDENCIES

Parallelize ready tasks, not wishful tasks



- Serialize changes to hot files and shared configuration
- Reserve one writer for the task board and Git integration
- Retest the combined result, not just individual branches

Sources: S10, S12

A claim is a protocol, not a mutex

- Claim: owner + paths + base revision + expiry
- Acknowledge before writing; release after handoff
- Expired or conflicting claims need coordinator review
- A Markdown task board cannot prevent filesystem writes

```
Task: search enhancement
Owner: worker A
Allowed paths: named module + named tests
Base: agreed commit / contract version
Shared files: integrator only
State: claimed -> running -> reviewed -> released
Conflict: STOP and report; do not overwrite
```

ENFORCEMENT

Use checks and permissions, not slogans

- Preflight + changed-path scope checks + required CI
- Tool restrictions reduce capability; terminal access is broad
- Hooks can inspect or deny calls; verify runtime behavior
- CODEOWNERS routes review; it does not lock edits

Sources: S5, S11, S12

Separate working copies for independent tasks

- Commit the contract and customizations before branching
- Each worktree: distinct files, branch and working index
- Ignored dependencies and uncommitted files are not copied
- Give servers and outputs distinct resources too

```
git status --short
git worktree add ..\triage-search -b lab-search HEAD
git worktree add ..\triage-summary -b lab-summary HEAD
git worktree list

# Use separate test outputs / server ports.
# Shared contract changes belong to the integrator.
```

Sources: S4, S15

INTEGRATION

A clean merge is not proof of correctness

- Review each task's diff, scope and test evidence
- Integrate one task at a time using normal Git / PR workflow
- Resolve conflicts against the frozen contract
- Rerun combined tests and the actual user flow

Sources: S4, S12

DEMO 3

Compare two safe orchestration patterns

- Pattern A: shared checkout, parallel readers, one writer
- Pattern B: committed contract + separate task worktrees
- Show a scope violation caught before integration
- PAYMENTS -> ISS-101 + ISS-105; export only active ISS-101

Sources: S4, S7, S12

SAFETY

Keep scope, data and approvals deliberate

- Use synthetic issues; never paste secrets into context
- Review scripts, MCP tools, plugins and acquired skills
- Worktree isolation != sandboxing != approval policy
- Review the diff before committing or sending external actions

Sources: S3, S8, S11, S15

EFFICIENCY

More agents is not automatically more value

- Use a direct request for small, cohesive work
- Delegate substantial independent work with a bounded result
- Limit depth and avoid duplicate investigation
- Choose available models for the task; measure quality and usage

Sources: S1, S7, S10, S12

PRACTICE APPLICATION

Issue triage board: small on purpose

- Node built-in HTTP + static page + node:test
- Fictional support issues; no database or API keys
- Eight issues: six active, two closed; four categories
- Passing starter + progressive features + separate answer key

LAB 1 / 30 MINUTES

Create and verify useful customizations

- Author short repository instructions
- Build a focused custom agent with narrow tools
- Build a skill with a supporting reference or script
- Capture discovery, invocation and honest result evidence

Sources: S5, S6, S8

LAB 2 / 35 MINUTES

Delegate a feature and collect proof

- Prepare a self-contained research brief
- Use a coordinator plus a read-only specialist
- Implement one exact behavior using the skill
- Verify boundary cases and request evidence-based review

Sources: S7, S9, S12

LAB 3 / 40 MINUTES

Orchestrate two tasks without collisions

- Freeze the contract and assign disjoint write sets
- Compare one-writer and worktree strategies
- Exercise the advisory board and a scope violation check
- Integrate serially, then validate the combined app

Sources: S4, S7, S10, S15

LAB 4 / 15 MINUTES

Review, hand off and reflect

- > Inspect all changed files and requirement coverage
- > Run the final quality gates and user-flow check
- > Write evidence, risks and remaining decisions
- > Explain where parallelism helped or hurt

```
OUTCOME / changed paths
CONTRACT / assumptions
CHECKS / exact commands and results
NOT RUN / reasons
RISKS / unresolved findings
INTEGRATION / reviewed state
NEXT DECISION / owner
```

Sources: S12

Stop, inspect, narrow and rerun

- Missing customization? Check path, harness and committed base
- Wrong tools? Check runtime inventory, not just YAML
- Overlapping edits? Pause writers and inspect actual files
- Stopping does not undo commands or external effects

Sources: S4, S6, S8, S12

TAKEAWAYS

Reliable orchestration is an engineering skill

- Role + recipe + task contract
- Separate context != isolated workspace
- Independent tasks + explicit ownership + one integrator
- Evidence beats confident prose

REFERENCE / NOT EXTRA TEACHING TIME

Official sources and availability

- S1-S3: Copilot app, Projects preview, sessions and permissions
- S4-S8: VS Code window, agent syntax, delegation and skills
- S9-S12: inheritance, /fleet, hooks preview and best practices
- S13-S15: 28 October release, compatibility and isolation

Sources: S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S13, S14, S15